Problem 5: Swap Nodes in Pairs (medium)

Problem Statement

Given a singly linked list, swap every two adjacent nodes and return the head of the modified list.

If the total number of nodes in the list is odd, the last node remains in place. Every node in the linked list contains a single integer value.

Examples

1. **Input:** `[1, 2, 3, 4]`
   Output: `[2, 1, 4, 3]`
   Justification: Pairs (1,2) and (3,4) are swapped.

2. **Input:** `[7, 8, 9, 10, 11]`
   Output: `[8, 7, 10, 9, 11]`
   Justification: Pairs (7,8) and (9,10) are swapped. 11 remains in its place as it has no adjacent node to swap with.

3. **Input:** `[5, 6]`
   Output: `[6, 5]`
   Justification: The pair (5,6) is swapped.

Try it yourself

Try solving this question here:

Java

```java
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }

class Solution {
    public ListNode swapPairs(ListNode head) {
        // TODO: Write your code here
        return head;
    }
}
```

Test Case | Run Code Result | Stdout

[1,2,3,4]

Use Example Testcases | Run | Submit

← Previous

Solution: Find If Doubly Linked List Is A Palindrome (Easy)

Next →

Solution: Swap Nodes In Pairs (Easy)

Mark as Completed